Context-Free Grammar

CFGs are more powerful than regular expressions. They are more powerful in the sense that whatever can be expressed using regular expressions can be expressed using context-free grammars, but they can also express languages that do not have regular expressions.

Context-Free Grammar

- A context-free grammar (CFG) G is a quadruple (V, Σ, R, S) where
- V: a set of non-terminal symbols
- Σ: a set of terminals ($V \cap \Sigma = \emptyset$)
- R: a set of rules ($R: V \rightarrow (V \cup \Sigma)^*$)
- S: a start symbol.

Context-Free Grammar

Example:

- $V = \{q, f,\}$
- $\Sigma = \{0, 1\}$
- $R = \{q \rightarrow 11q,\ \ q \rightarrow 00f,$
  $f \rightarrow\ 11f,\ \ f \rightarrow \varepsilon\ \ \}$
- $S = q$
- $(R= \{q \rightarrow 11q \mid 00f,\ f \rightarrow 11f \mid \varepsilon\ \})$

- If $S \Rightarrow \cdots \Rightarrow t$, then we write $S \overset{*}{\Rightarrow} t$.
- A string x in Σ^* is generated by $G=(V,\Sigma,R,S)$ if $S \Rightarrow^* x$.
- $L(G) = \{\ x \text{ in } \Sigma^* \mid S \Rightarrow^* x\}$.

Context-Free Grammar

Example

- $G = (\{S\}, \{0,1\}, \{S \rightarrow 0S1 \mid \varepsilon\}, S)$
- ε in $L(G)$ because $S \Rightarrow \varepsilon$.
- 01 in $L(G)$ because $S \Rightarrow 0S1 \Rightarrow 01$.
- 0011 in $L(G)$ because

  $S \Rightarrow 0S1 \Rightarrow 00S11 \Rightarrow 0011$.
- 0^n1^n in $L(G)$ because $S \Rightarrow^* 0^n1^n$.
- $L(G) = \{0^n1^n \mid n \geq 0\}$

- A language L is context-free if there exists a CFG G such that $L = L(G)$.

Context-Free Grammar

Construct a context free grammar for palindromes containing 0s and 1s.

G= ({P},{0,1},A, P)

Where A is defined by the set of rules

$P \Rightarrow \epsilon$

$P \Rightarrow 0$

$P \Rightarrow 1$

$P \Rightarrow 0P0$

$P \Rightarrow 1P1$

Context-Free Grammar

Construct a context free grammar for all integers (with sign)

$G = \{V, \Sigma, R, S\}$

$V = \{S, G, D, I\}$, $\Sigma = \{0, 1, 2, 3, \ldots, 9, +, -\}$

$S \Rightarrow GI$
$G \Rightarrow +|-$
$I \Rightarrow DI \mid D$
$D \Rightarrow 1|2|3|\ldots|9\ |0$

Derivation Trees

Let G=({S, A}, {a,b}, P, S), where P consists of

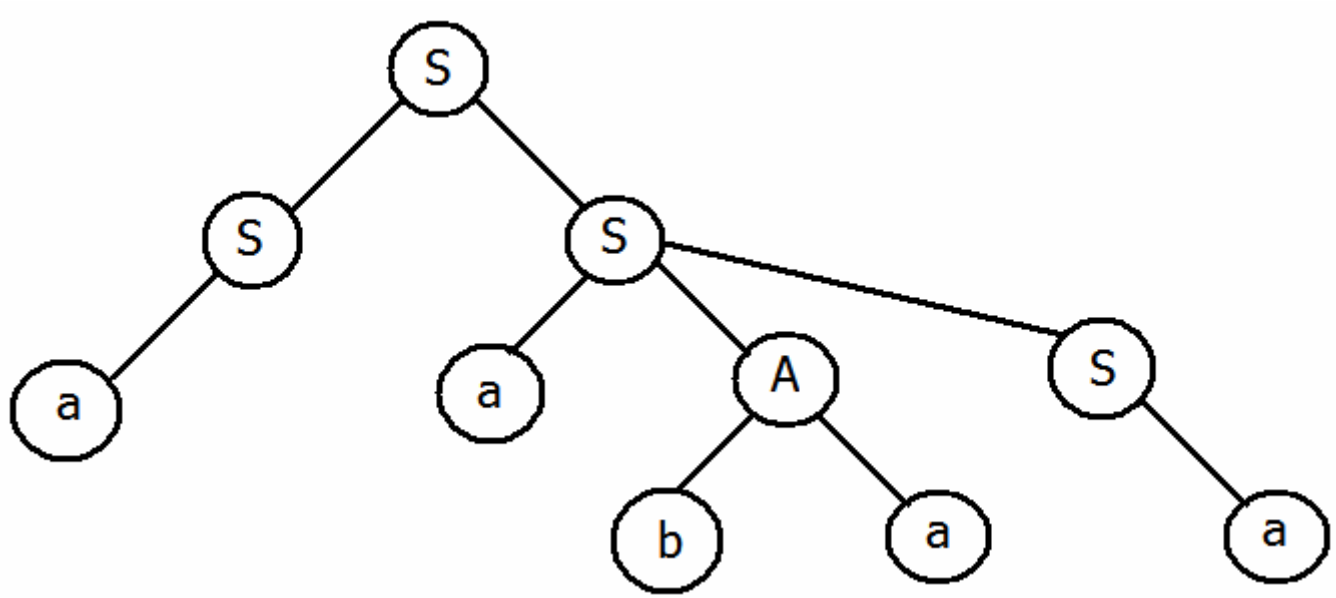
$S \Rightarrow aAS \mid a \mid SS$
$A \Rightarrow SbA \mid ba$

Derive the string aabaa

$S \Rightarrow SS \Rightarrow aS \Rightarrow aaAS \Rightarrow aabaS \Rightarrow aabaa$

Construct a derivation tree for aabaa

Derivation Trees



Derivation Trees

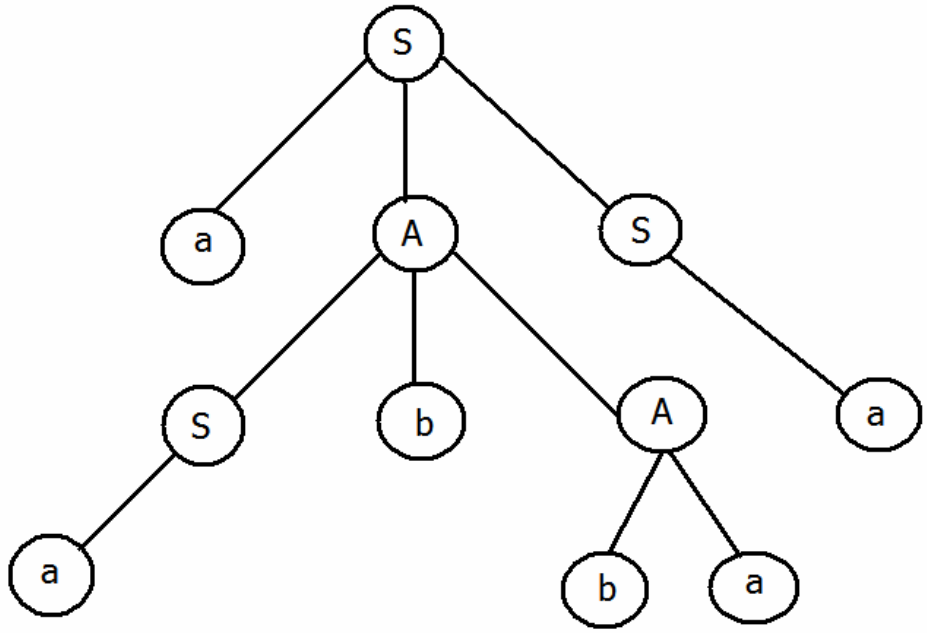
Consider G whose productions are

$S \Rightarrow aAS \mid a$

$A \Rightarrow SbA|SS|ba$

Show that the derivation of aabbaa is possible

$$S \Rightarrow aAS \Rightarrow aSbAS \Rightarrow aabAS \Rightarrow aabbaS \Rightarrow aabbaa$$



Derivation Trees

A parser knows the grammar of the programming language

The parser finds the derivation of a particular input

input

10 + 2 * 5

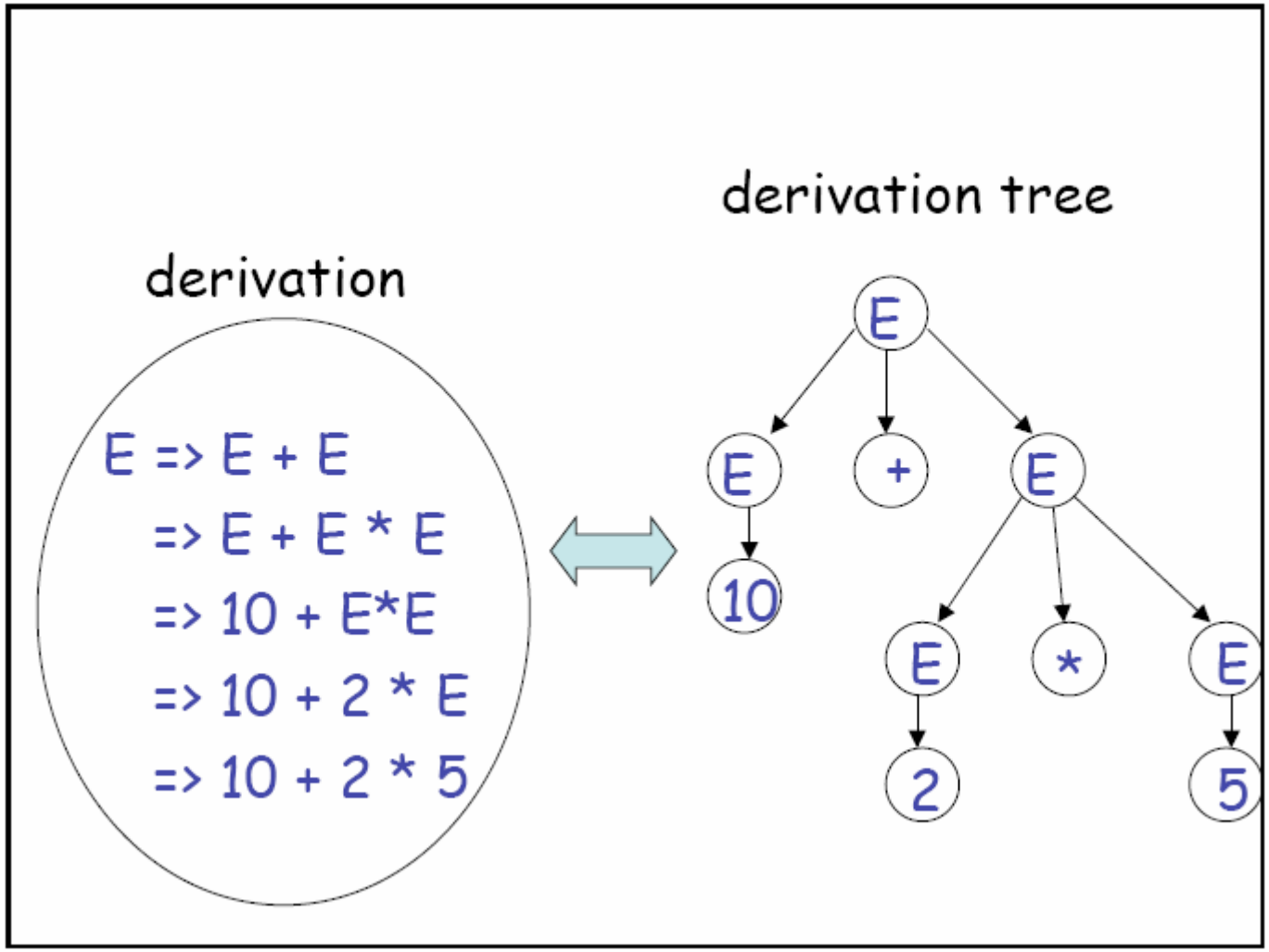
Parser

```
E -> E + E
     | E * E
     | INT
```

derivation

```
E => E + E
  => E + E * E
  => 10 + E*E
  => 10 + 2 * E
  => 10 + 2 * 5
```

Derivation Tree



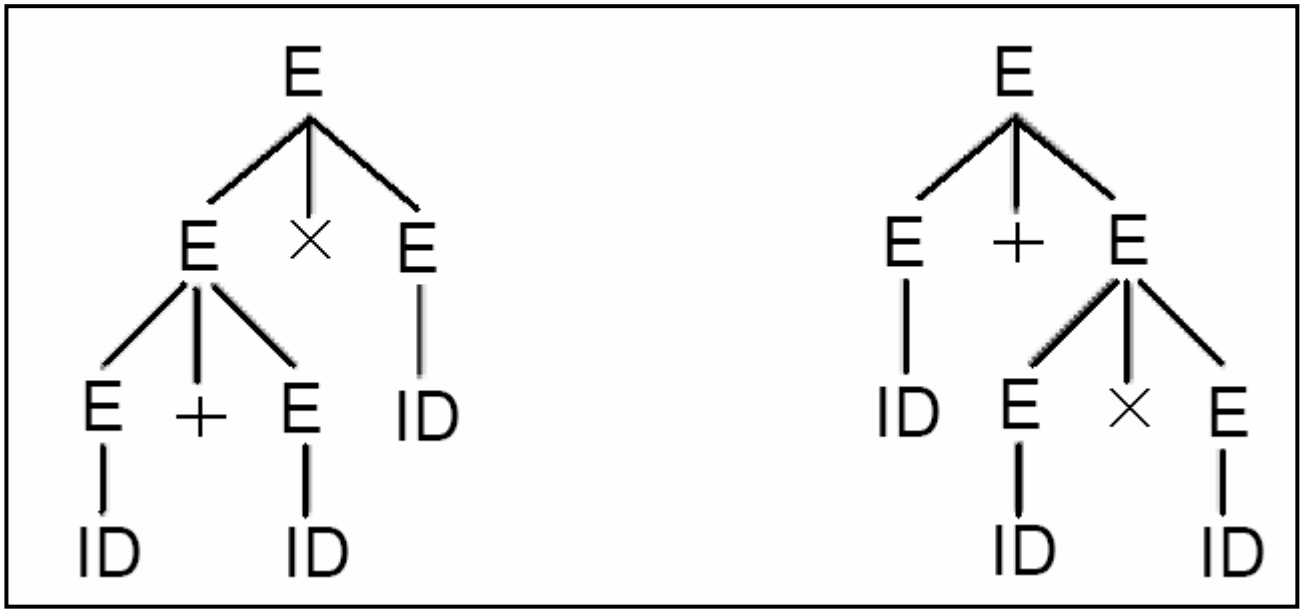
Ambiguity

Consider the grammar for arithmetic expressions involving addition and multiplication operators:

$E \rightarrow E+E$
$E \rightarrow E \times E$
$E \rightarrow ID$

Consider the sentence ID+ID×ID.

This can be parsed in two different ways.



Ambiguity

In the figure, the parse tree to the left gives the addition operator precedence over multiplication. In other words, an expression such as $3+5\times 9$ is evaluated as $(3+5)\times 9$ with a result of 72. Whereas, the tree to the right, does what is considered the standard practice in programming languages, i.e. giving $\times$ precedence over +. The previous example would be evaluated as $3+(5\times 9)$ resulting in 48.

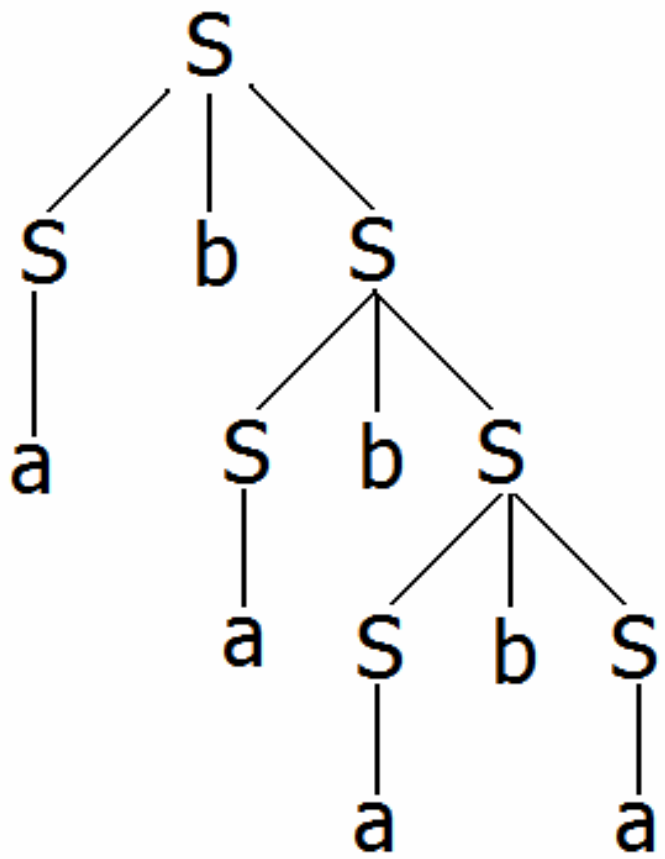
A context free grammar G is ambiguous if there exists some $w \in L(G)$, which is ambiguous.

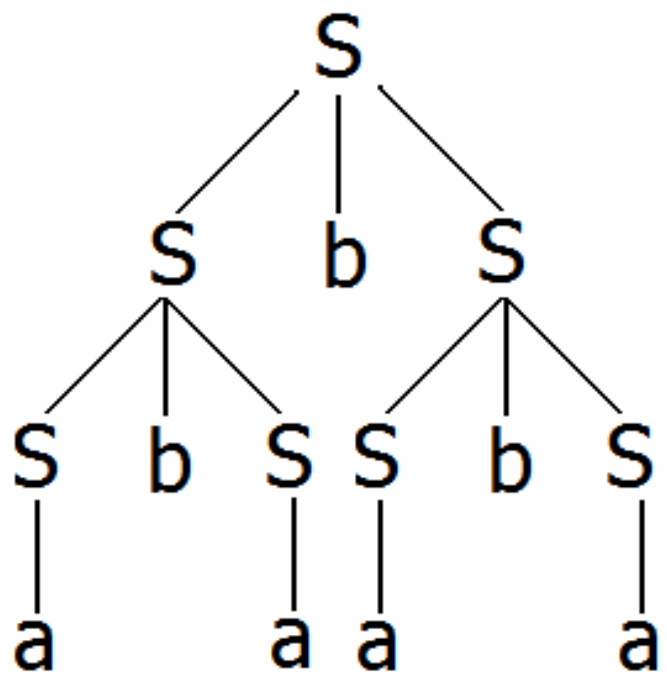
If G is the grammar $S \rightarrow SbS|a$

Show that G is ambiguous.

Ambiguity

To prove that G is ambiguous, we need to find a w $\in$ L(G), which is ambiguous . Consider the word abababa.





Ambiguity

Definition 1 *A CFG G is said to be ambiguous if there exists some $w \in L(G)$ that has at least two distinct parse trees.*

Definition 2 *A CFL L is said to be inherently ambiguous if all its grammars are ambiguous.*

Definition 3 *If L is a CFL for which there exists an unambiguous grammar, then L is said to be unambiguous.*

Even one grammar for L is unambiguous, then L is an unambiguous language.

Good news: Sometimes we can remove ambiguity by hand.

Bad news: There is no algorithm to do it.

More bad news: Some CFL's have only ambiguous CFG's.

Removing Ambiguity

Let us consider the grammar:

$$
\begin{aligned}
E &\rightarrow I \mid E + E \mid E * E \mid (E) \\
I &\rightarrow a \mid b \mid Ia \mid Ib \mid I0 \mid I1
\end{aligned}
$$

What about a + a $\times$ a ?

Solution: We introduce more variables, each representing expressions that share a level of "binding strength".

1. A factor is an expression that cannot be broken apart by an adjacent $*$ or $+$. Our factors are

   (a) Identifiers
   (b) A parenthesized expression

2. A term is a product of one or more factors.

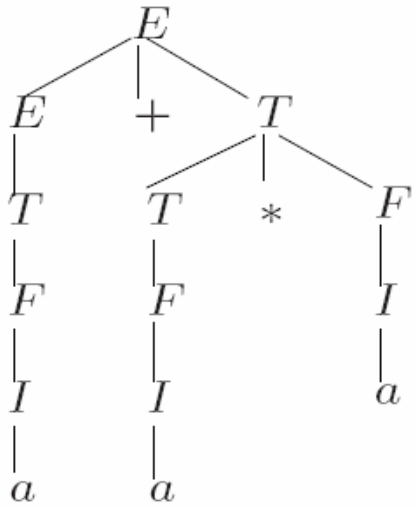
3. The rest are expressions, i.e., they can be broken apart by $*$ or $+$.

Removing Ambiguity

We will let F stand for factors, T for terms, and E for expressions. Consider the following grammar:

$$
\begin{aligned}
I &\rightarrow a \mid b \mid Ia \mid Ib \mid I0 \mid I1, \\
F &\rightarrow I \mid (E), \\
T &\rightarrow F \mid T * F, \\
E &\rightarrow T \mid E + T.
\end{aligned}
$$

Now the only parse tree for $a + a * a$ will be



Simplification of CFG

If G is a CFG , then we can construct a simplified equivalent CFG G′ with the help of following steps:

1. Eliminate all null productions to get G_1.
2. Eliminate all unit productions in G_1 to get G_2.
3. Construct a reduced Grammar G′ from G_2.

Simplification of CFG

1.Elimination of null productions

Let $G_1=(V, \Sigma, P', S)$ be the GFG having NO null productions.

Consider the Grammar G whose productions are given below. Construct a Grammar G_1 without null productions generating $L(G)-\{\varepsilon\}$

$S \rightarrow aS \mid AB$
$A \rightarrow \varepsilon$
$B \rightarrow \varepsilon$
$D \rightarrow b$

Simplification of CFG

Step 1: Construction of the set W of all nullable variables

$W_1=\{A_1 \in V \mid A_1 \rightarrow \varepsilon$ is a production in P$\}=\{A,B\}$

$W_{i+1}= W_i \cup \{K \in V \mid$ there exists a production $K \rightarrow \alpha$ with $\alpha \in W_i^* \}$

$W_2=\{A,B\} \cup \{S\}$ as $S \rightarrow AB$ is a production with $AB \in W_1^*$

$= \{ S,A,B\}$

$W_3= W_2 \cup \varnothing = W_2$

Erasing from RHS

Construction of P': $D \rightarrow b$, $S \rightarrow aS$, $S \rightarrow AB$, $S \rightarrow a$, $S \rightarrow A$,

$S \rightarrow B$.

Simplification of CFG

2.Elimination of unit productions

A unit production in a context free grammar G is a production of the form $A \rightarrow B$ where A and B are variables(non terminals) in G.

For such a variable A the following steps have to be applied:

Step1: Construction of the set of variables derivable from A.

$W_0(A)=\{A\}$

$W_{i+1}(A)=W_i(A) \cup \{B \in V \mid C \rightarrow B \text{ is in } P \text{ with } C \in W_i(A)\}$

Simplification of CFG

Step2: Construction of A productions

The A-productions in G_2 are either

(a) The non unit production in G_1 or

(b) $A \rightarrow \alpha$ whenever $B \rightarrow \alpha$ is in G_1 with $B \in W(A)$ and $\alpha \notin V$.

Now we can define $G_{2,}$ where P_2 is constructed using step2 for every $A \in V$

Simplification of CFG

Example:

Let the productions in G_1 be

$S \rightarrow AB$, $A \rightarrow a$, $B \rightarrow C|b$, $C \rightarrow D$, $D \rightarrow E$ and $E \rightarrow a$

$W(B)=\{B,C,D,E\}$, $W(C)=\{C,D,E\}$, $W(D)=\{D,E\}$

The productions P_2 in G_2 are:

$S \rightarrow AB$, $A \rightarrow a$, $B \rightarrow a|b$, $C \rightarrow a$, $D \rightarrow a$ and $E \rightarrow a$

Simplification of CFG

3.Construction of reduced Grammars:

Many productions in P may not be useful for the purpose of derivation.

It would be better to eliminate (1) variables that do not derive any terminal string and (2) symbols that are not reachable from the start symbol.

If G is a CFG, then we can find an equivalent grammar G' such that each variable in G' derives some terminal string.

Let G2=(V, Σ ,P,S). We can define G' =(V', Σ ,P',S) as follows.

Simplification of CFG

a) Construction of V′:

We define $W_i \subseteq V$ by recursion.

$W_1=\{A \in V|$ there exists a production $A \rightarrow \omega$ where $\omega \in \Sigma^*\}$

$W_{i+1}=W_i \cup \{A \in V|$ there exists some production $A \rightarrow \alpha$ with $\alpha \in (\Sigma \cup W_i)^*\}$

At some point $W_k=W_{k+1}$. Then we get $V'=W_k$.

b) Construction of P′.

$P'=\{A \rightarrow \alpha \mid A, \alpha \in (V' \cup \Sigma)^*\}$

Simplification of CFG

Example: Let G be

$S \rightarrow AB$, $A \rightarrow a$, $B \rightarrow b$, $B \rightarrow C$, and $E \rightarrow c$

Find G' such that every variable in G' derives some terminal string.

Here we get $V'=\{S,A,B,E\}$.

So $P'= S \rightarrow AB$, $A \rightarrow a$, $B \rightarrow b$ and $E \rightarrow c$

Simplification of CFG

If $G=(V, \Sigma, P, S)$, we can construct an equivalent grammar $G'= (V', \Sigma', P', S)$ such that every symbol in $V' \cup \Sigma'$ is derivable from S.

a) Construction of Wi

$W_1=\{S\}$.

$W_{i+1}=W_i \cup \{X \in (V \cup \Sigma) \mid$ there exists a production $A \rightarrow \alpha$ with $A \in W_i$ and α containing the symbol $X\}$

At some point $W_k=W_{k+1}$.

b) Construction of V', Σ', P'.

$V'=V \cap W_k$, $\Sigma'= \Sigma \cap W_k$, $P'=\{A \rightarrow \alpha \mid A \in W_k\}$

Simplification of CFG

Example: Consider G=({S,A,B,E}, {a,b,c},P,S), where P consists of $S \rightarrow AB$, $A \rightarrow a$, $B \rightarrow b$ and $E \rightarrow c$.

$W_3=\{S,A,B\} \cup \{a,b\}$

$V'= \{S,A,B\}$, $\Sigma' =\{a,b\}$

$P'= S \rightarrow AB$, $A \rightarrow a$, $B \rightarrow b$

Simplification of CFG

Find reduced grammars equivalent to the following

1) $S \rightarrow AB|CA$, $B \rightarrow BC|AB$, $A \rightarrow a$, $C \rightarrow aB|b$.

2) $S \rightarrow aAa$, $A \rightarrow Sb|bCC|DaA$, $C \rightarrow abb|DD$, $E \rightarrow aC$, $D \rightarrow aDA$.

Answers:

1) $S \rightarrow CA$, $A \rightarrow a$, $C \rightarrow b$

2) $S \rightarrow aAa$, $A \rightarrow Sb|bCC$, $C \rightarrow abb$

Normal Forms for CFG

In a context free grammar, the R.H.S of a production can be any string of variables and terminals. When the production in G satisfy certain restrictions, then G is said to be in a 'normal form'.

1. Chomsky Normal Form(CNF)

In CNF we have restrictions on the length of R.H.S and the nature of symbols in the R.H.S

A CFG G is said to be in CNF, if every production is of the form $A \rightarrow a$, or $A \rightarrow BC$, and $S \rightarrow \varepsilon$. When ε is in L(G) we assume that S does not appear on the R.H.S of any production.

Normal Forms for CFG

Example: Consider a CFG whose productions are

$S \rightarrow AB|\varepsilon$, $A \rightarrow a$, $B \rightarrow b$

Is it in CNF? **YES**

Reduction to CNF

Consider an example. Let G be

$S \rightarrow ABC|aC$, $A \rightarrow a$, $B \rightarrow b$, $C \rightarrow c$.

Except $S \rightarrow ABC|aC$, all other productions are in the form required for CNF. The terminal a in aC can be replaced by a new variable D. By adding a new production $D \rightarrow a$. So $S \rightarrow aC$ becomes $S \rightarrow DC$.

Normal Forms for CFG

$S \rightarrow ABC$ is not in the required form. So it can be replaced by two productions: $S \rightarrow AE$ and $E \rightarrow BC$.

It is important to note that unit productions and null productions are to be removed before substitutions.

Elimination of terminals on R.H.S: If there is a terminal a_i on the R.H.S of a production, then add a new variable (non terminal) $C_{ai} \rightarrow a_i$

Restricting the number of variables on R.H.S: Consider productions of the form $A \rightarrow A_1A_2....A_m$. We introduce new productions $A \rightarrow A_1C_1$, $C_1 \rightarrow A_2C_2$, ..., $C_{m-2} \rightarrow A_{m-1}A_m$

Normal Forms for CFG

Example1: Reduce the following grammar G to CNF:

$S \rightarrow aAD$, $A \rightarrow aB|bAB$, $B \rightarrow b$, $D \rightarrow d$.

Answer: $S \rightarrow C_aC_1$, $A \rightarrow C_aB|C_bC_2$, $C_1 \rightarrow AD$,
$C_2 \rightarrow AB$, $B \rightarrow b$, $D \rightarrow d$, $C_a \rightarrow a$, $C_b \rightarrow b$.

Example2: Reduce the following grammar G to CNF:

$S \rightarrow aAbB$, $A \rightarrow aA|a$, $B \rightarrow bB|b$.

Answer: $S \rightarrow C_aC_1$, $C_1 \rightarrow AC_2$, $C_2 \rightarrow C_bB$, $A \rightarrow C_aA$,

$B \rightarrow C_bB$, $C_a \rightarrow a$, $C_b \rightarrow b$, $A \rightarrow a$, $B \rightarrow b$.

Normal Forms for CFG

Greibach Normal Form(GNF):

A grammar is in GNF if every production is of the form

$A \rightarrow a\alpha$, where $\alpha \in V^*$. And $a \in \Sigma$.

$S \rightarrow \varepsilon$ is in G, when ε is in L(G) and we assume that S does not appear on the R.H.S of any production.

Example: $S \rightarrow aAB|\varepsilon$, $A \rightarrow bC$, $B \rightarrow b$, $C \rightarrow c$ is in GNF.

Normal Forms for CFG

Lemma1: Let G be a CFG, and $A \rightarrow B\lambda$ be a production in P. Assume that P also has the following productions:
$B \rightarrow \beta_1|\beta_2|....|\beta_s$

We can define $P_1=(P - \{A\rightarrow B\lambda\}) \cup \{A\rightarrow \beta_i\lambda|1\leq i \leq s\}$

Then G_1 having P_1 is also a CFG equivalent to G.

Example: Consider G with the following productions:

$A \rightarrow Bab$, $B \rightarrow aA|bB|aa|AB$

G_1 which is equivalent to G can be constructed with the following productions.

$A \rightarrow aAab|\ bBab|aaab|ABab$,

$B \rightarrow aA|bB|aa|AB$

Normal Forms for CFG

Lemma2: Let G contains productions of the form

$A \rightarrow A\alpha_1 | A\alpha_2 | .... | A\alpha_r | \beta_1 | \beta_2 | .... | \beta_s$ (β_i's do not start with A)

Let Z be a new variable in G_1, where P_1 is defined as follows:

$A \rightarrow \beta_1 | \beta_2 | .... | \beta_s$

$A \rightarrow \beta_1 Z | \beta_2 Z | .... | \beta_s Z.$

$Z \rightarrow \alpha_1 | \alpha_2 | .... | \alpha_r$

$Z \rightarrow \alpha_1 Z | \alpha_2 Z | .... | \alpha_r Z$

Example: Consider G with the following productions:

$A \rightarrow aBD | bDB | c | AB | AD$

Normal Forms for CFG

Then G_1, equivalent to G contains the following productions:

$A \rightarrow aBD|bDB|c$

$A \rightarrow aBDZ|bDBZ|cZ$

$Z \rightarrow B|D$

$Z \rightarrow BZ|DZ$

Normal Forms for CFG

Reduction to GNF

Step1: Check whether the given grammar is in CNF. If not in CNF, make it in CNF. Rename the variables as $A_1, A_2, \ldots A_n$. With $S=A_1$.

Step2: A_i productions should be of the form $A_i \rightarrow A_j\lambda$ such that $i < j$. If there are productions of the form $A_i \rightarrow A_i\lambda$, apply Lemma2 to get rid of such productions. Otherwise apply Lemma1.

Step3: Modify Z_i productions. Apply Lemma1 to eliminate productions of the form $Z_i \rightarrow A_k\lambda$.

Normal Forms for CFG

Example1: Construct a grammar in GNF equivalent to the grammar: $S \rightarrow AA|a$, $A \rightarrow SS|b$

Answer: The given grammar is in CNF. Let S be A_1 and A be A_2.

$A_1 \rightarrow A_2A_2$

$A_1 \rightarrow a$

$A_2 \rightarrow A_1A_1$

$A_2 \rightarrow b$

A_1 productions are in the required form. $A_2 \rightarrow b$ is also in the required form.

Normal Forms for CFG

Apply Lemma1 to $A_2 \rightarrow A_1A_1$.

$$A_2 \rightarrow A_2A_2A_1, \qquad A_2 \rightarrow aA_1, \qquad A_2 \rightarrow b.$$

We need to apply Lemma2 to $A_2 \rightarrow A_2A_2A_1$. Let Z_2 be the new variable.

$$A_2 \rightarrow aA_1, \qquad A_2 \rightarrow b$$

$$A_2 \rightarrow aA_1Z_2, \qquad A_2 \rightarrow bZ_2$$

$$Z_2 \rightarrow A_2A_1, \qquad Z_2 \rightarrow A_2A_1Z_2.$$

A_2-productions are $A_2 \rightarrow aA_1 \mid b \mid aA_1Z_2 \mid bZ_2$.

Now we can eliminate $A_1 \rightarrow A_2A_2$ using Lemma1.

Normal Forms for CFG

$$A_1 \rightarrow a \mid aA_1A_2 \mid bA_2 \mid aA_1Z_2A_2 \mid bZ_2A_2$$

The Z_2-productions to be modified are

$$Z_2 \rightarrow A_2A_1,\ Z_2 \rightarrow A_2A_1Z_2.$$

We apply Lemma1 to get

$$Z_2 \rightarrow aA_1A_1 \mid bA_1 \mid aA_1Z_2A_1 \mid bZ_2A_1$$

$$Z_2 \rightarrow aA_1A_1Z_2 \mid bA_1Z_2 \mid aA_1Z_2A_1Z_2 \mid bZ_2A_1Z_2$$

Normal Forms for CFG

P_1 consists of

$$A_1 \rightarrow a \mid aA_1A_2 \mid bA_2 \mid aA_1Z_2A_2 \mid bZ_2A_2$$

$$A_2 \rightarrow aA_1 \mid b \mid aA_1Z_2 \mid bZ_2$$

$$Z_2 \rightarrow aA_1A_1 \mid bA_1 \mid aA_1Z_2A_1 \mid bZ_2A_1$$

$$Z_2 \rightarrow aA_1A_1Z_2 \mid bA_1Z_2 \mid aA_1Z_2A_1Z_2 \mid bZ_2A_1Z_2$$

Normal Forms for CFG

Example2: Convert the following grammar into GNF.

$$S \to AB,\ A \to BS \mid b,\ B \to SA \mid a$$

Rename S, A, B as A_1, A_2, A_3, respectively.

The productions are :

$$A_1 \to A_2A_3,\ A_2 \to A_3A_1 \mid b,\ A_3 \to A_1A_2 \mid a.$$

The A_1-production $A_1 \to A_2A_3$ is in the required form.
The A_2-productions $A_2 \to A_3A_1 \mid b$ are in the required form.
$A_3 \to a$ is in the required form.

Apply Lemma1 to $A_3 \to A_1A_2$.

Normal Forms for CFG

The resulting productions are $A_3 \rightarrow A_2A_3A_2$.
Applying the lemma once again to $A_3 \rightarrow A_2A_3A_2$,
we get $A_3 \rightarrow A_3A_1A_3A_2 \mid bA_3A_2$.

The A_3-productions are $A_3 \rightarrow a \mid bA_3A_2$ and $A_3 \rightarrow A_3A_1A_3A_2$.
we have to apply Lemma 2 to $A_3 \rightarrow A_3A_1A_3A_2$

$$A_3 \rightarrow a \mid bA_3A_2, \qquad A_3 \rightarrow aZ_3 \mid bA_3A_2Z_3$$

$$Z_3 \rightarrow A_1A_3A_2, \qquad Z_3 \rightarrow A_1A_3A_2Z_3$$

The A_3-productions are

$$A_3 \rightarrow a \mid bA_3A_2 \mid aZ_3 \mid bA_3A_2Z_3$$

Normal Forms for CFG

eliminate $A_2 \rightarrow A_3A_1$ using Lemma 1

$$A_2 \rightarrow aA_1 \mid bA_3A_2A_1 \mid aZ_3A_1 \mid bA_3A_2Z_3A_1$$

$$A_2 \rightarrow b \mid aA_1 \mid bA_3A_2A_1 \mid aZ_3A_1 \mid bA_3A_2Z_3A_1$$

We apply Lemma 1 to $A_1 \rightarrow A_2A_3$ to get

$$A_1 \rightarrow bA_3 \mid aA_1A_3 \mid bA_3A_2A_1A_3 \mid aZ_3A_1A_3 \mid bA_3A_2Z_3A_1A_3$$

The Z_3-productions to be modified are:

$$Z_3 \rightarrow A_1A_3A_2 \mid A_1A_3A_2Z_3$$

Normal Forms for CFG

We apply Lemma 1 and get

$$Z_3 \rightarrow bA_3A_3A_2 \mid bA_3A_3A_2Z_3$$

$$Z_3 \rightarrow aA_1A_3A_3A_2 \mid aA_1A_3A_3A_2Z_3$$

$$Z_3 \rightarrow bA_3A_2A_1A_3A_3A_2 \mid bA_3A_2A_1A_3A_3A_2Z_3$$

$$Z_3 \rightarrow aZ_3A_1A_3A_3A_2 \mid aZ_3A_1A_3A_3A_2Z_3$$

$$Z_3 \rightarrow bA_3A_2Z_3A_1A_3A_3A_2 \mid bA_3A_2Z_3A_1A_3A_3A_2Z_3$$